

TABLE MEMORY · STATE REPRODUCIBILITY

1. 为什么需要“有记忆的表”

能查到当前数据，不等于能复现当时回答。

从 Week03 的 ingest correctness，过渡到 Week04 的 table state reproducibility。

开场定调

坏案例复盘

可查 vs 可回看

状态证据

Repo 边界

行业信号

Week04 学习链

01

Memory

为什么 AI 数据工程需要“有记忆的表”

02

State Model

snapshot、manifest、metadata log 与 time travel

03

Bronze / Silver

Week03 入湖如何站成最小 4 表

04

Pylceberg

Catalog、Warehouse、写入、历史查看与 Schema Evolution

05

Baseline

files / history / snapshots 视角下的 Week04 验收

这节课先解决什么问题

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

为什么它重要

- Week03 已经解决数据怎样稳定进入系统、失败后怎样 replay / backfill。
- 但没有解决：答案变了、评测漂了、坏案例出现时，如何证明是哪一版数据状态导致。
- raw bucket、Postgres 当前表、向量库都能查，但不一定能回看当时状态。
- 本课先建立“有记忆的表”的团队复盘心智。

学完这节课，你至少应该能做到

- 解释“能查当前数据”和“能回看当时数据状态”的差异。
- 用 Northstar bad case 说明为什么需要 snapshot 锚点。
- 区分 raw bucket、Postgres 当前表、pgvector、Iceberg table 的角色。
- 写出 lakehouse_foundation_v1.md 与 state_memory_questions_v1.md。

本课产出：docs/blueprints/week04/lakehouse_foundation_v1.md + state_memory_questions_v1.md。

先把 Week04 的地图看清

本周一句话：从 ingest correctness 升级到 table state reproducibility。

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline



Week04 不是组件周，而是 AI 数据工程第一次把“状态记忆”落到数据实体层。

LECTURE DIAGRAM · REDRAW

Week04 总览主线图：从 ingest 到状态记忆

基于讲义总图重绘：Week03 的 ingest / replay / backfill 进入 Week04 的可回看表状态，再支撑后续 transform、RAG、eval 与 release。



PART 01

先把“状态记忆”讲清

先分清能查、能回看、能绑定评测/发布，再谈 Iceberg 机制。

Lesson 01

表不存在不是最危险；
状态说不清才危险。

snapshot

history

release

eval

AI 项目里真正危险的不是表不存在，而是状态不可复现

你有数据、有索引、有答案，但说不清它们到底对应哪一版状态，这才是生产复盘里的大坑。

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

当前可查

只能说明“现在有什么”。

历史可回看

能说明“当时发生了什么”。

状态可绑定

索引、评测、release 才有锚点。

Northstar Edge Gateway: 同一问题答案变了, 但谁也说不清为什么

01

周一回答

建议回滚到 5.7.9

02

周三回答

建议替换硬件

03

系统都能查

raw / PG / vector / prompt /
log

04

但无 snapshot

状态无法绑定

05

复盘失败

只能猜数据是否变了

屏幕上看起来像什么

- raw bucket 能查
- Postgres 当前表能查
- 向量索引有结果
- Prompt 仓库也有版本

实际已经坏在哪

- 周一基于哪版文档资产说不清
- 周三索引重建消费哪批文档说不清
- 评测分数变化到底在数据前还是索引后说不清

FOUR OBJECTS

四类数据对象对比：谁能承担“状态账本”？

不要把能查的对象都当成可复现数据状态。

对象	能查当前？	能回看历史状态？	适合承担什么角色
raw bucket	能，但通常要靠路径/清单	弱；除非额外版本化和 manifest	文件柜：保存原始输入
PostgreSQL 当前表	能	弱；默认只代表当前业务视图	业务视图：当前事实查询
pgvector / 向量索引	能召回	弱；索引通常不等于数据状态账本	索引：服务检索，不负责状态记忆
Iceberg snapshot-able table	能	强；snapshot/history/files 可检查	数据账本：提交历史与状态锚点

“可查” vs “可回看”

“可查” 解决现在有什么；“可回看” 解决当时发生了什么。

场景	只可查当前数据	可回看表状态
当前查询	能回答今天表里有什么	也能回答今天表里有什么
历史回看	需要靠日志和备份猜	可绑定 snapshot / history
评测绑定	只能说 “可能数据变了”	可绑定具体 table snapshot
索引重建	只知道索引重建过	知道索引消费哪版文档资产
release 复盘	代码和 prompt 有版本，数据缺锚点	release 绑定代码 + prompt + table snapshot

有记忆的表到底记住什么

至少要留下 5 类状态证据。

01 提交后的表状态

每次 append / overwrite 形成状态变更。

02 当前版本由哪些 files 构成

不是靠目录猜，而是由 metadata 组织。

03 snapshot 之间关系

能看历史和当前。

04 schema / partition 演进

字段与布局变化要可解释。

05 可引用状态锚点

time travel / baseline / release 可以绑定。

Iceberg 不等于自动解决所有问题

先把边界说清，避免把 Week04 误解成万能湖仓方案。

不自动修数据质量

坏数据写进去，snapshot 只是帮你记住坏在哪一版。

不自动让 RAG 变准

它给索引提供状态锚点，不替代切分、检索、评测。

不替代 Week02 contract

Contract 仍负责什么能进、什么必须拦。

不替代 eval / tracing

状态锚点只是评测和追踪的必要输入。

不自动完成治理回滚

回滚需要策略、审批、runbook 和验证。

团队复盘必须问的 8 个问题

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

01 raw 文档版本?

这次回答基于哪版 raw 文档。

02 ingest batch?

对应哪次 ticket/doc ingest。

03 Iceberg snapshot?

数据状态锚点是什么。

04 index version?

索引消费了哪版数据。

05 Prompt version?

提示词状态是否变化。

06 eval drift point?

分数变化发生在哪个状态之后。

07 business boundary?

字段或权限是否变化。

08 rollback target?

能否定位到稳定状态。

从 ingest 到有记忆的数据系统

Week03 的 manifest/state/replay 进入 Iceberg Bronze/Silver, 最终支撑索引一致性、评测和发布复盘。



Week04 的目标不是多一个存储层, 而是让数据状态第一次可命名、可回看、可验收。

MENTAL MODEL

文件柜、业务视图、索引、数据账本：四个比喻帮小白记住

比喻不是严谨定义，但能帮助学生快速建立边界。

比喻	对应对象	记住一句话
文件柜	raw bucket	原始材料在，但不自动成为可复盘状态
业务视图	Postgres 当前表	适合查询当前事实，不保证历史状态
搜索目录	pgvector / index	服务召回，不负责解释数据提交历史
数据账本	Iceberg table	每次提交都形成可检查状态证据

QUIETLY FAIL · INDEX WITHOUT SNAPSHOT

索引没有数据 snapshot 绑定，会让bad case复盘变成猜谜会

01

文档更新

raw doc 新版本进入

02

索引重建

vector 重算

03

评测下降

某类 query 变差

04

无 snapshot

不知道消费哪版数据

05

复盘猜谜

只能对比日志

屏幕上看起来像什么

- 索引存在
- 搜索也有结果
- 评测只是下降，没有明显报错

实际已经坏在哪

- 索引与文档资产状态没有绑定
- 无法判断是数据变了、切分变了、Prompt 变了还是模型变了
- 后续 Week08/11 会持续遇到这个坑

先建立状态锚点，不迁移全生产

最小闭环只要求这些证据能被解释和演示。

表状态可以提交

append / overwrite 之后形成新 snapshot。

历史状态可以查看

snapshots / history / metadata log 能 inspect。

旧状态可以回看

time travel 有明确目标。

字段可边界内演进

Week04 只做 add-column 级别演示。

baseline 可记录

当前状态被写成报告，而不是截图。

要对齐哪些 repo 对象

不伪造命令，不伪造运行结果；先明确对象和结构化占位。

pipelines/lakehouse/iceberg_schemas.py

最小 4 表 schema 的 source of truth。

pipelines/lakehouse/assets.py

后续 Dagster thin wrapper / asset 入口。

infra/docker-compose.yml

PostgreSQL、MinIO、Dagster 等本地基线。

pyproject.toml

Pylceberg / PyArrow 等依赖边界。

runbooks/week04/ + reports/week04/

运行路径与 baseline report 的落点。

本课工程入口：先写设计工件，不急着伪造命令

如果项目命令尚未落地，只保留结构化目标动作。

TERMINAL · STRUCTURED HANDOFF

```
# Week04 Lesson01 输出工件
mkdir -p docs/blueprints/week04 reports/week04 runbooks/week04

touch docs/blueprints/week04/lakehouse_foundation_v1.md
touch docs/blueprints/week04/state_memory_questions_v1.md

# 待 Week4 项目代码落地后，同步 pipelines.lakehouse.* 的真实命令。
```

先写判断

Week04 为什么存在、解决什么、不解决什么。

再写复盘问题

坏案例需要问哪些状态问题。

不要伪造命令

真实命令等项目代码落地后同步。

官方资料怎么支撑本课判断

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

Iceberg Reliability

有效 snapshots 存在 table metadata file 中，当前 snapshot 有引用；提交通过 atomic metadata swap。

可靠读与历史

读者始终使用一致 snapshot；snapshots 可保留为 history 并支持 rollback。

Pylceberg

Python implementation 可在无需 JVM 的情况下访问 Iceberg tables。

课程收口

这些不是炫技能力，而是坏案例复盘和 release-aware 数据系统的状态底座。

PRODUCTION HEURISTIC

生产判断口令：坏案例能不能绑定到数据状态？

遇到答案变了，不要先问模型有没有换；先问状态是否可绑定。

问数据

文档 / 工单是否发生过状态提交？

问索引

索引消费哪版文档资产？

问评测

分数变化绑定哪个 snapshot？

问发布

这次 release 绑定哪些数据状态？

问回滚

要回到哪个数据状态，而不只是回滚代码？

Lesson01 的两份工作怎么写

lakehouse_foundation_v1.md

说明 Week04 为什么存在、解决什么、不解决什么。

state_memory_questions_v1.md

列出坏案例复盘时必须问的状态问题。

写作原则

每一句都要能连接到后续 snapshot / history / baseline。

不要写成口号

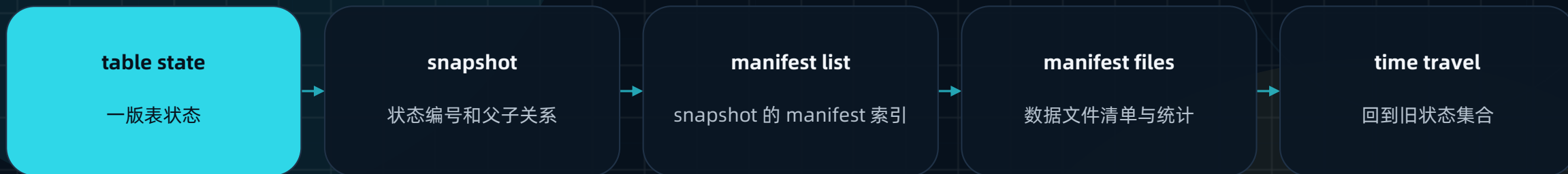
不要只说“需要可追溯”，要写“要绑定哪个状态对象”。

本课最常见的 5 个误解

误解	为什么危险	修正
Iceberg 是大数据平台，和 AI/RAG 无关	忽略索引、评测、release 都需要数据状态	把 Iceberg 讲成表状态记忆层
有对象存储就等于 Lakehouse	只有文件，没有提交历史和状态解释	需要 table metadata / snapshots
有向量库就能复现回答	向量索引不是数据账本	索引绑定文档资产 snapshot
time travel 就是复制很多份表	误解机制，导致成本和风险判断错	回到旧 snapshot 状态集合
Week04 做完就完成治理	过度承诺	治理、发布、权限仍在后续周次

Iceberg 靠什么记住表状态?

下一讲从判断进入机制：snapshot、manifest、metadata file、metadata log 与 time travel。



snapshot 不是一个文件，manifest 不是目录清单。

把“状态记忆”立成 Week04 的核心主线

本课最重要的 8 个判断

- 能查当前数据，不等于能复现当时回答。
- raw bucket / Postgres 当前表 / 向量库都不是统一数据状态账本。
- Week04 的核心是 table state reproducibility。
- 有记忆的表至少要有 snapshot、history、metadata、files、schema evolution 证据。
- Iceberg 不自动解决数据质量和 RAG 正确性。
- 本周先求本地可跑、可解释、可验收。
- 后续索引、评测、release 都需要绑定数据状态。
- 下一讲进入 Iceberg 状态模型。

下一讲

Iceberg 状态模型：snapshot、manifest、metadata log 与 time travel。

课后最小行动

把今天的判断写进 repo 工件；不要只截图证明“看过”。

SNAPSHOT MODEL · TIME TRAVEL

2. Iceberg 的状态模型

Time travel 不是复制旧表，
而是回到旧 snapshot 所代表的状态集合。

把 snapshot、manifest list、manifest files、metadata log 与 data files 讲成一条状态证据链。

snapshot

manifest

metadata log

time travel

schema evolution

retention

Week04 学习链

01

Memory

为什么 AI 数据工程需要 “有记忆的表”

02

State Model

snapshot、manifest、metadata log 与 time travel

03

Bronze / Silver

Week03 入湖如何站成最小 4 表

04

Pylceberg

Catalog、Warehouse、写入、历史查看与 Schema Evolution

05

Baseline

files / history / snapshots 视角下的 Week04 验收

这节课先解决什么问题

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

为什么它重要

- 上一讲只立了“为什么需要状态记忆”。
- 这一讲要讲清 Iceberg 靠什么把一组 files 组织成可提交、可回看、可演进的表状态。
- 很多人把 Iceberg 误解成按日期分目录的升级版，这会直接误导 time travel 和 evolution。
- 我们用 metadata inspection 的顺序把状态链讲清楚。

学完这节课，你至少应该能做到

- 解释 snapshot 记录的是什么。
- 说清 manifest list / manifest 怎样指向 data files。
- 解释 metadata file、metadata log、history 为什么是状态证据链。
- 说清 time travel 为什么不是复制整张表。
- 解释 schema evolution 与 snapshot 模型的边界。

本课产出：snapshot_state_model_v1.md + time_travel_demo_notes.md。

Lesson02 在 Week04 里的位置

从 Memory 的需求，进入 Iceberg 的状态模型，再交给最小 4 表设计。

01 Memory

02 State Model

03 Bronze/Silver

04 PyIceberg

05 Baseline



机制不是为了炫技，而是为了坏案例复盘能说清“当时是哪版表状态”。

LECTURE DIAGRAM · REDRAW

current metadata pointer → snapshot → manifest → files

基于讲义图重绘：从当前 metadata pointer 进入 metadata.json，再到 snapshot list / manifest list / manifest files / data files。



Time travel

可以选择 old snapshot S1。

Metadata log

记录 metadata file 演进。

Data files

Parquet 文件不是靠目录猜。

PART 01

先把状态对象拆清楚

snapshot、manifest、metadata log、history、files 不是一个概念。

Lesson 02

Iceberg 用 metadata 把一次表状态提交组织成可追踪对象。

snapshot

manifest

metadata

files

Iceberg 不是按日期分目录的升级版

它用 metadata 把一次表状态提交组织成可追踪对象；这才让 time travel、schema evolution 和 reliable reads 成立。

01 Memory

02 State Model

03 Bronze/Silver

04 PyIceberg

05 Baseline

不是目录

不是 `s3://table/date=...` 的升级。

是提交历史

每次写入形成一版状态。

可追踪对象

snapshot → manifest → files。

Git 类比：有帮助，但必须知道边界

Git 里	Iceberg 里	类比边界
commit	snapshot	记录表状态，不是代码 diff
commit history	metadata log / history	记录状态演进，不是开发分支历史
文件清单	manifest / manifest list	记录 data files 和统计信息
工作目录文件	data files	文件本身不等于表状态
checkout old commit	time travel old snapshot	回到旧状态集合，不是复制旧表

STATE OBJECTS

状态对象拆分：每个对象回答一个问题

对象	小白理解	工程上解决什么
table metadata file	表的状态说明书	存 schema、snapshots、current snapshot 等
snapshot	一版表状态	让一次提交可命名、可回看
manifest list	snapshot 的 manifest 索引	不用读完所有 manifest 才规划
manifest	数据文件清单 + 统计	帮助确定哪些 files 属于状态
data file	实际 Parquet 文件	承载数据，不单独代表表状态
metadata log	metadata 文件演进记录	支撑状态证据链
history	谁何时成为 current	支撑复盘和 baseline

Snapshot 不是一个文件

它是一版可命名、可引用、可回看的表状态。

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

不是目录

不要把 snapshot 当成 date=xxx。

不是单个数据文件

它指向一组 metadata 和 data files。

写入会改变状态

append / overwrite / schema change 后形成新提交。

可复盘能力

能否解释写入影响，取决于 snapshot 与 metadata 证据。

Manifest 不是目录清单

它是当前状态如何组织 data files 的核心证据之一。

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

包含哪些 data files

当前状态引用的实际文件。

文件属于哪些分区

帮助查询剪枝和回放边界。

文件级统计

record_count、size、lower/upper bounds 等。

状态变化入口

没有 manifest, time travel 会退化成文件猜谜。

METADATA POINTER

current metadata pointer: 可靠读的入口

Iceberg 的可靠性不靠列目录猜，而靠 metadata pointer 和 atomic swap。

valid snapshots

存在 table metadata file 中。

current snapshot

metadata file 里保存当前 snapshot 引用。

atomic swap

提交替换 current table metadata file path。

可靠读

读者使用一致 snapshot，无需持锁。

Time travel 为什么不是复制整张表

它依赖稳定 snapshot id 和未过期的状态证据链。

snapshot id

能指定旧状态。

manifest list

旧 snapshot 指向它的 manifest。

manifest + files

最终确定旧状态引用哪些 files。

retention boundary

旧 snapshot 过期后，time travel 范围会缩短。

silver.ticket_fact 的 snapshot 时间线示例

同一张表的不同 snapshot 可以被 Week08 索引、Week11 评测、Week14 发布治理引用。



表状态一旦可命名，后续索引、评测和发布就不再只绑定代码。

Metadata inspection 应该怎么看

观察顺序: snapshots → history → files → metadata log。



不要只看“表能读”，要看状态证据链能不能解释。

FIELD GUIDE

常见 metadata inspection 字段怎么讲

字段	回答什么	讲课口径
snapshot_id	是哪一版状态	后续 index/eval/release 可绑定
parent_id	上一版状态是谁	能构成状态时间线
made_current_at	什么时候成为 current	坏案例复盘定位时间点
operation	append / overwrite / replace	解释状态变化类型
manifest_list	snapshot 指向的 manifest list	从状态到文件清单的桥
file_path	实际 data file 路径	文件证据入口
record_count / file_size	数据规模与小文件信号	baseline 的核心指标
latest_schema_id	当前 schema 版本	schema evolution 证据

NOT DATE FOLDERS

为什么不是按日期分目录

能力	按日期目录	Iceberg 状态模型
历史回看	靠路径和文件列表猜	绑定 snapshot
原子提交	很难保证全表状态一致	metadata atomic swap
schema 演进	目录无法表达字段版本	metadata change
查询剪枝	依赖手工目录约定	manifest/statistics 辅助规划
release/eval 绑定	只能绑定路径或时间	绑定 snapshot / history

Schema evolution 边界：支持不等于随便改

Week04 只做 add-column 级别演示，不提前展开复杂兼容性。

支持的变更

add / drop / rename / update / reorder。

metadata change

schema updates 不需要重写 data files。

课程边界

Week04 只演示 add-column。

不要过度承诺

复杂兼容性、权限语义、下游消费要后续治理。

HIDDEN PARTITIONING

Hidden partitioning 与 partition evolution 先埋伏笔

Lesson03 会展开，这里先建立状态模型。

hidden partitioning

查询者不必直接依赖具体分区布局。

partition evolution

已有表可更新 partition spec；旧数据布局保持不变。

metadata operation

分区演进不急切重写 files。

工程判断

布局变化要进入 baseline report。

Snapshot 保留与清理边界

能 time travel 多久，取决于 snapshot 和 metadata 保留策略。

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

每次写入

会创建新 snapshot / version。

time travel / rollback

依赖有效 snapshot。

expire snapshots

可控制 metadata 和文件规模。

过早过期

旧状态不再可用于 time travel。

并发写入与 atomic swap

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

optimistic concurrency

写者基于当前状态写新 metadata。

atomic swap

提交时替换 metadata file path。

retry validation

冲突后检查假设是否仍成立。

reliable reads

读者使用一致 snapshot，无需持锁。

INCIDENT · EXPIRED SNAPSHOT

过早清理 snapshot, 会让 bad case 复盘失去旧状态

01

持续写入

snapshot 快速增长

02

盲目清理

expire snapshots

03

旧状态消失

time travel 失败

04

评测回看

无法复现旧结果

05

团队争论

到底谁改坏了

屏幕上看起来像什么

- metadata 变小了
- 存储压力暂时下降
- 看起来维护动作成功

实际已经坏在哪

- 旧 snapshot 不再可用于 time travel
- baseline report 没记录保留策略
- release / eval 回看能力被缩短

官方资料怎么支撑本课判断

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

Reliability

valid snapshots 存在 metadata file; current snapshot 有引用; 提交通过 atomic operation。

Evolution

schema evolution 是 metadata change; partition evolution 也可以作为 metadata operation。

Maintenance

expire snapshots 可控 metadata 与文件规模, 但会缩短 time travel 范围。

Performance

manifest list / manifest files 支撑 scan planning 与 metadata filtering。

本课最容易误读的 5 件事

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

误读	为什么错	正确口径
snapshot 是文件	把状态和文件混了	snapshot 是一版表状态
manifest 是目录清单	低估 metadata 的作用	manifest 记录 data files + stats
time travel 复制旧表	误判成本和机制	回到旧 snapshot 状态集合
schema evolution 随便改	忽略兼容性和下游消费	先做 add-column 边界演示
cleanup 越早越好	缩短可回看历史	先有保留策略和 baseline

Lesson02 自检清单

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

snapshot 是什么?

能说成一版表状态。

manifest 做什么?

能说出 data files + stats。

time travel 靠什么?

能说出 snapshot → manifests → files。

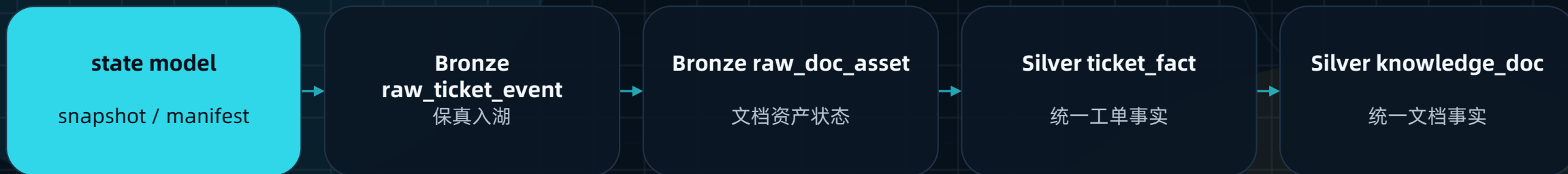
schema evolution 边界?

能说出 metadata change 与 add-column demo。

cleanup 风险?

能说出过期后不可 time travel。

状态模型如何落到 Bronze / Silver 最小表？



表越多不代表越生产级，最小 4 表先真实可写、可回看。

把 Iceberg 讲成表状态模型

01 Memory

02 State Model

03 Bronze/Silver

04 PyIceberg

05 Baseline

本课最重要的 8 个判断

- Iceberg 不是日期目录升级版。
- snapshot 是一版表状态，不是单个文件。
- manifest 是状态索引，不是目录清单。
- time travel 回到旧状态集合，不是复制整表。
- metadata inspection 要按 snapshots → history → files → metadata log 看。
- schema evolution 有边界，不等于随便改字段。
- snapshot 保留策略会影响可回看能力。
- 下一讲把状态模型落到 Bronze / Silver 最小表设计。

下一讲

Bronze / Silver 最小 4 表设计与 hidden partitioning。

课后最小行动

把今天的判断写进 repo 工件；不要只截图证明“看过”。

BRONZE · SILVER · HIDDEN PARTITIONING

3. Bronze / Silver 最小表设计

表越多不代表越生产级；
状态型表真实可写、可回看，才是第一版 Lakehouse 的能力。

把 Week03 入湖基线落成 bronze.raw_ticket_event、bronze.raw_doc_asset、silver.ticket_fact、silver.knowledge_doc。

最小 4 表

Bronze / Silver

mapping

schema source

hidden partitioning

边界

Week04 学习链

01

Memory

为什么 AI 数据工程需要 “有记忆的表”

02

State Model

snapshot、manifest、metadata log 与 time travel

03

Bronze / Silver

Week03 入湖如何站成最小 4 表

04

Pylceberg

Catalog、Warehouse、写入、历史查看与 Schema Evolution

05

Baseline

files / history / snapshots 视角下的 Week04 验收

这节课先解决什么问题

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

为什么它重要

- 当前项目里，到底哪些表要先真实存在，哪些对象现在还不该展开。
- 第一版 Lakehouse 失败常常不是表太少，而是最小闭环没站稳，层数却先堆起来。
- source-to-Iceberg mapping 必须先写，否则 coding agent 很容易在字段名、nullable、时间语义、去重键上跑偏。

学完这节课，你至少应该能做到

- 解释为什么本周只做最小 4 表。
- 说清 Bronze、Silver、Gold 的正确角色。
- 写出 source-to-Iceberg mapping 的最小字段。
- 解释 hidden partitioning 和 partition evolution 的边界。
- 明确本周不交付哪些表。

本课产出：source_to_iceberg_mapping_v1.md + bronze_silver_table_design_v1.md。

LECTURE DIAGRAM · REDRAW

ticket 和 document 两条流如何进入最小 4 表

基于讲义图重绘：ticket ingest 进入 raw_ticket_event / ticket_fact；document ingest 进入 raw_doc_asset / knowledge_doc。



最小 4 表和字段映射，更复杂的 Gold、指标和消费层

第一版 Lakehouse 的能力，不是表的数量，而是这些表能真实写入、形成 snapshot、被回看、被验收。

先最小

2 张 Bronze + 2 张 Silver。

先 mapping

人和 agent 对齐字段语义。

先状态

真实可写、可 inspect、可 baseline。

PART 01

先把最小表设计站住

不要一上来设计一堆 Gold / mart / serving 表。

Lesson 03

表越多不代表越生产级；
最小闭环才是第一版能力。

。

Bronze

Silver

mapping

boundary

为什么本周只做最小 4 表

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

避免 scope creep

Week05+ 才进入 transform / semantic layer。

避免层数先堆起来

最小闭环没站稳，表越多越乱。

先观察字段语义

source 语义、分区策略、主键先看清。

减少返工

过早铺十几张表会让 mapping 大量返工。

FOUR TABLE MAP

最小 4 表设计地图

表名	层级	从哪里来	解决什么问题	后续消费
bronze.raw_ticket_event	Bronze	Week03 ticket ingest	保留工单原始事件和 replay 入口	silver.ticket_fact / Week05
bronze.raw_doc_asset	Bronze	MinIO raw docs / doc ingest	保留文档资产与版本状态	silver.knowledge_doc / Week08
silver.ticket_fact	Silver	raw_ticket_event	统一工单事实与状态	Week05 semantic layer
silver.knowledge_doc	Silver	raw_doc_asset	统一知识文档资产状态	Week08 retrieval consistency

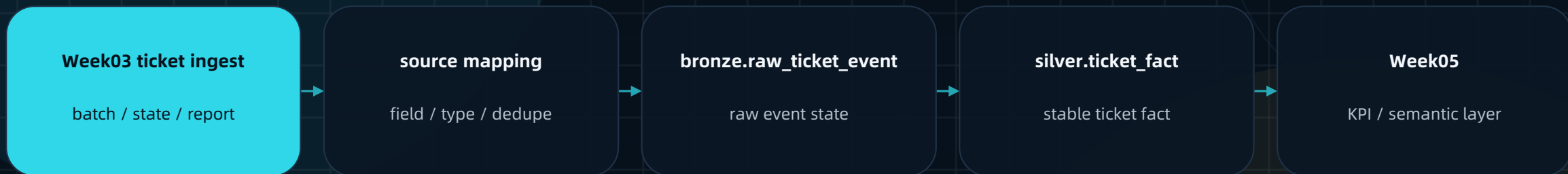
Bronze / Silver / Gold 的正确角色

层级	正确角色	本周边界
Bronze	尽量保真的入湖记录；保留 ticket/doc 输入状态与 replay 入口	做 raw_ticket_event、raw_doc_asset
Silver	稳定可消费的业务对象；统一工单事实和文档资产状态	做 ticket_fact、knowledge_doc
Gold	语义层、指标层、服务层和最终消费接口	Week05+ 再展开，不抢跑

TICKET FLOW

ticket flow: 从 Week03 baseline 到 Bronze / Silver

工单流先形成事件保真，再形成可消费事实。

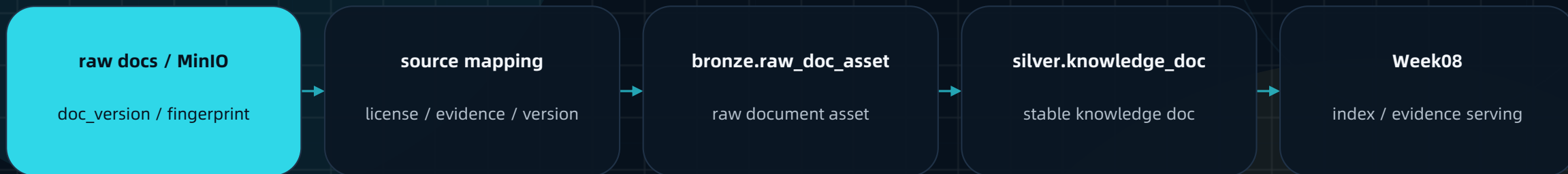


ticket_fact 不是 blind append，它要表达一个稳定可消费的工单事实状态。

DOCUMENT FLOW

document flow: 从 raw docs 到 knowledge_doc

文档流保留资产状态，为后续 Week08 retrieval consistency 绑定数据版本。



向量索引不是数据状态账本，knowledge_doc snapshot 才是文档状态锚点。

MAPPING FIRST

Source-to-Iceberg mapping 为什么必须先写

coding agent 最容易在字段名、nullable、时间语义和去重键上跑偏。

source object

来源对象是谁。

source field → target field

字段怎么映射。

transform / type

是否转换、类型如何对齐。

required / missing handling

缺失时怎么处理。

dedupe / idempotency

重复输入和重跑怎么解释。

MAPPING FIELDS

mapping 至少包含哪些字段

字段	回答什么	缺失风险
source object	来自哪个源对象	agent 随意猜源
source field	源字段名是什么	字段名误写
target table / field	进入哪张表哪个字段	目标表混乱
transform	是否派生 / cast / normalize	时间语义被误改
required	缺失是否允许	nullable 随意扩散
dedupe key	如何识别同一业务事件	重复写入
idempotency strategy	重跑是否有副作用	不可解释重复

SCHEMA SOURCE OF TRUTH

Schema source of truth: 不要随意发明字段

真实字段以 repo、contract 和 ingest 输出为准。

iceberg_schemas.py

最小 4 表 schema 的项目 source of truth。

实际 DDL / source 表定义

避免课程页面随意造字段。

Week02 data contract

准入约束仍然有效。

Week03 manifest / ingest 输出

批次和状态信息要延续。

命令未落地时

只写设计边界，不写死字段全集。

HIDDEN PARTITIONING

Hidden partitioning 解决什么

查询者按逻辑字段思考，表格式处理 partition transform 与 pruning。

传统目录分区

查询者和路径布局绑死。

Iceberg hidden partitioning

分区 transform 由表格式维护。

查询写逻辑字段

不直接依赖具体目录布局。

scan planning

manifest list / manifest stats 辅助剪枝。

DIRECTORY VS HIDDEN

手工目录 vs hidden partitioning

这不是路径风格问题，是状态演进与查询边界问题。

维度	手工目录分区	Iceberg hidden partitioning
查询写法	容易依赖目录字段	写逻辑字段，由 Iceberg 利用分区信息
布局演进	改布局往往要新表/改查询	partition evolution 可共存旧新布局
回放边界	靠路径约定	可绑定 metadata / partition spec
baseline 记录	容易只记录路径	记录 table + snapshot + files + partition distribution

Partition evolution 的边界

分区字段必须服务查询模式和回放边界，不是为了高级感。

当前策略

服务本地可跑 baseline。

不要过度分区

小数据集盲目分区只会增加复杂度。

服务查询模式

产品线、时间、source 类型要有理由。

演进要记录

spec 变化进入 baseline report。

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

NOT THIS WEEK

本周明确不交付什么

对象	本周状态	原因
bronze.raw_ticket_event	必须物化	Ticket Bronze 最小闭环
bronze.raw_doc_asset	必须物化	Document Bronze 最小闭环
silver.ticket_fact	必须物化	Ticket Silver 最小闭环
silver.knowledge_doc	必须物化	Document Silver 最小闭环
knowledge_section	只预留边界	Week07/08 再展开
evidence_anchor	只说明不抢跑	后续 evidence serving
support_kpi_mart	不做	Week05+ 指标层
kb_serving_asset	不做	Serving/RAG 周再做

DESIGN MISTAKES

学员常见设计错误

这些错误会让最小闭环变成漂亮但不可复盘的模型。

错误	表面上	真正问题
Bronze 里提前业务解释	看起来语义清楚	丢失 replay 和原始证据
Silver blind append 当前状态表	写入最简单	当前事实不可解释地重复/漂移
分区字段和查询无关	显得高级	scan 和回放边界没帮助
用技术字段替代业务键	容易实现	业务 dedupe / idempotency 不稳
课程页面随意发明字段	PPT 很完整	repo 落地会返工
schema 文件出现的表都物化	表很多	scope creep 到 Week05+

INCIDENT · BRONZE OVER-INTERPRETED

Bronze 过早做业务解释，后面 replay 反而失真



屏幕上看起来像什么

- Bronze 看起来很“干净”
- Silver 写起来更省事
- 第一次 demo 很顺

实际已经坏在哪

- 保真入口丢了
- 原始事件状态不可复盘
- replay/backfill 无法判断是源数据问题还是 transform 问题

INCIDENT · SILVER BLIND APPEND

Silver 盲目 append 当前状态，会把事实表变成历史垃圾堆

01

每批 ticket 更新

append 一批

02

没有 upsert 语义

同一 ticket 多行

03

query 当前事实

取到旧状态

04

KPI 漂移

统计不稳

05

baseline 难读

snapshot 不能解释事实

屏幕上看起来像什么

- 写入总是成功
- snapshot 也在增长
- row count 看起来更丰富

实际已经坏在哪

- Silver 不是 raw event dump
- 当前事实需要稳定 key 与语义
- 重复和历史状态要能解释

官方资料怎么支撑本课判断

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

Hidden partitioning

查询不需要写特定分区布局，Iceberg 根据分区信息自动剪枝。

Partition evolution

已有表可以更新 partition spec，旧数据保持旧布局，新数据进入新布局。

Metadata filtering

manifest list acts like index over manifest files，减少扫描不必要 metadata。

课程收口

分区不是炫技，它服务查询模式、回放边界和 baseline。

WEEK05 / WEEK08 HANDOFF

这 4 表怎么接 Week05 和 Week08

最小 4 表不是终点，而是后续 transform、semantic layer 和 retrieval consistency 的状态输入。



后续不是从零开始，而是消费 Week04 的稳定表状态。

如何用 Pylceberg 跑出本地最小闭环？

表设计站住之后，下一讲进入 Catalog、Warehouse、写入、history / snapshots / files inspection。



先让核心机制可见，再谈更复杂平台接入。

把状态模型落到最小 4 表设计

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

本课最重要的 8 个判断

- Week04 先站住最小 4 表。
- Bronze 保真，Silver 统一。
- Gold 不在本周抢跑。
- mapping 必须先于 table create 和 materialization。
- hidden partitioning 不是手工目录分区。
- partition evolution 是工程判断，不是炫技。
- 字段以 repo schema / contract / ingest 输出为准。
- 下一讲跑 Pylceberg 本地最小闭环。

下一讲

Pylceberg 本地最小闭环：Catalog、Warehouse、写入、历史查看与 Schema Evolution。

课后最小行动

把今天的判断写进 repo 工件；不要只截图证明“看过”。

PYICEBERG · LOCAL CLOSED LOOP

4. Pylceberg 本地最小闭环

先让核心机制可见，再谈更复杂的平台接入。

用 Pylceberg + PostgreSQL-backed SQL Catalog + MinIO warehouse 跑出可演示、可验收的最小闭环。

catalog

warehouse

materialize

inspect

schema evolution

thin wrapper

Week04 学习链

01

Memory

为什么 AI 数据工程需要 “有记忆的表”

02

State Model

snapshot、manifest、metadata log 与 time travel

03

Bronze / Silver

Week03 入湖如何站成最小 4 表

04

Pylceberg

Catalog、Warehouse、写入、历史查看与 Schema Evolution

05

Baseline

files / history / snapshots 视角下的 Week04 验收

这节课先解决什么问题

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

为什么它重要

- Week04 前 3 课讲清了判断、状态模型和 4 表设计。
- 这一讲开始收口到本地可跑闭环。
- 我们要讲清 catalog 怎样加载、warehouse 怎样声明、表怎样 ensure/create、数据怎样最小 materialize。
- 还要解释 Dagster 为什么本周只做 thin wrapper。

学完这节课，你至少应该能做到

- 解释为什么当前路线选择 Pylceberg + PostgreSQL SQL Catalog + MinIO。
- 区分 catalog / warehouse / table location。
- 描述一次最小 create / append / inspect / add-column 演示。
- 写出 catalog_runtime_plan_v1.md 与 runbooks/week04/README.md。
- 说清不伪造命令与结构化占位的边界。

本课产出：catalog_runtime_plan_v1.md + runbooks/week04/README.md + schema_evolution_demo_notes.md。

LECTURE DIAGRAM · REDRAW

devbox / thin wrapper → Pylceberg → SQL Catalog + MinIO

基于讲义图重绘：CLI 或 Dagster thin wrapper 进入 Pylceberg，连接 SQL Catalog 与 warehouse，形成 Bronze/Silver tables。



PostgreSQL 不是存所有 Iceberg 数据文件；MinIO 也不是普通文件堆。

Week04 不是重型基础设施

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

先用 Pylceberg + PostgreSQL SQL Catalog + MinIO warehouse 让核心机制可见、可跑、可验收，再谈平台全家桶。

轻量

复用现有 compose。

可见

能看到 catalog / warehouse / files。

可验收

snapshots / history / files 进入 report。

ROUTE CHOICE

为什么当前课程先选这条路线

路线	本地复杂度	能否覆盖 Week04 目标	是否推荐
Pylceberg + PostgreSQL SQL Catalog + MinIO	中	覆盖 catalog / warehouse / write / inspect / evolution	推荐
Pylceberg + SQLite Catalog	低	适合探索，但与多容器教学基线不完全一致	不作为主线
REST Catalog	中高	覆盖 catalog 机制，但服务排错成本高	暂不推荐
Spark + Hive/Nessie/Trino	高	覆盖更多生产场景，但会抢走状态模型注意力	不作为 Student Core 主线

WHY NOT HEAVY STACK

为什么不先引入 Spark / Hive / Nessie / Trino

组件	为什么不是本周主线	正确时机
Spark	把注意力带到分布式计算	后续规模化场景
Hive Metastore	本地最小闭环负担偏重	平台工程扩展
Nessie	引入 catalog branching, 超出 Week04	release/branching 语义后续讲
Trino	把重点带到查询服务和 SQL engine	查询层和 serving 后续展开
REST Catalog	学生本地环境排错成本偏高	平台化后再考虑

Catalog / Warehouse / Table Location 必须拆开

Catalog

记录表在哪里、metadata 在哪里。

Warehouse

Iceberg metadata 和 data files 的存储根。

Table Location

某张表在 warehouse 里的具体位置。

常见误解

PostgreSQL 不存所有数据文件；
MinIO 不是普通文件堆。

ENV CONFIG

配置必须走 env，不要写死在课程代码里

配置项	小白解释	写错会怎样
catalog type	告诉 Pylceberg 用哪类 catalog	加载方式错误
catalog uri	PostgreSQL SQL Catalog 连接串	namespace / table 读写失败
warehouse root	Iceberg 默认存储根	文件落错 bucket
s3 endpoint	MinIO / S3-compatible endpoint	本机能跑，容器里失败
access key / secret	warehouse 凭证	写文件失败或权限错误
namespace	bronze / silver 逻辑空间	表注册到错误空间

最小 materialization I/O

基于讲义图重绘：PostgreSQL / Week03 baseline 进入 mapping，再由 PyIceberg 写入 Bronze/Silver 并产出可 inspect 证据。

01 Memory

02 State Model

03 Bronze/Silver

04 PyIceberg

05 Baseline



关键不是“写入成功”，而是写入后形成哪个 snapshot，inspect 输出是否能证明表状态。

MATERIALIZE STEPS

一次 materialize 背后至少发生 10 件事

这页解释命令背后的工程动作。

01 读取 env 与配置

catalog / warehouse / endpoint。

02 加载 SQL Catalog

load catalog。

03 ensure namespace

bronze / silver。

04 ensure table schema

以 source of truth 为准。

05 读取输入

Week03 源表或对象存储。

06 按 mapping 转换

Arrow / writable data。

07 append / overwrite

形成新状态。

08 产生 snapshot

进入历史。

09 inspect

history / files / snapshots
。

10 输出 report

写入可验收证据。

MINIMUM DEFENSES

Dry-run / plan / validation / idempotency

这些防线决定 Week04 是工程闭环，不是脚本碰巧跑过。

防线	作用	缺失后果
dry-run	先展示计划与影响，不直接污染表状态	试错直接写坏表
materialization plan	明确 source、target、schema、dedupe key	agent 随意猜字段
schema validation	写入前确认字段/类型/nullable	写入后才发现 schema 错
deterministic dedupe	同一输入重复到达结果可预测	重复不解释
idempotency	同一次 materialization 重跑无不可解释副作用	重跑越写越脏

DAGSTER THIN WRAPPER

为什么 Dagster 本周只做 thin wrapper

CLI / devbox 先可跑

核心逻辑先放可直接调用的 Python module。

不要假设依赖自动进入容器

pyproject.toml 改了不等于 Dagster 容器就可用。

Week06 再资产工厂化

asset factory / partition / backfill 后续系统讲。

本周重点

Pylceberg table state, 而不是 orchestration 厚包装。

PyIceberg API : 只讲本周要用的动作

目标	PyIceberg API / 概念	本课用途
load catalog	load_catalog	加载 SQL Catalog
create namespace	create_namespace / list_namespaces	确保 bronze / silver 逻辑空间
create table	catalog.create_table	ensure 最小 4 表 schema
load table	catalog.load_table	定位已有表
append / overwrite	table.append / table.overwrite	形成新 snapshot
scan	table.scan().to_arrow()	验证数据可读
inspect	inspect.history / snapshots / files	写 baseline report
evolution	update_schema().add_column()	add-column demo

ENGINEERING HANDOFF

当前命令如何写：结构化占位，不伪造真实路径

真实命令以 Week4 项目代码落地后同步为准。

TERMINAL · STRUCTURED HANDOFF

待 Week4 项目代码落地后，同步 pipelines.lakehouse.* 的真实命令。

目标动作：

1. check catalog/env

2. ensure namespace/tables

3. materialize bronze/silver

4. inspect snapshots/history/files

5. run add-column schema evolution demo

当前可以先写：

touch docs/blueprints/week04/catalog_runtime_plan_v1.md

touch runbooks/week04/README.md

touch reports/week04/schema_evolution_demo_notes.md

只讲目标动作

不要在 PPT 里伪造可执行命令。

看观察点

catalog、warehouse、snapshot、inspect。

等代码落地

再同步真实 pipelines.lakehouse.* 命令。

常见错误排查：先定位是哪一层坏了

错误现象	可能原因	排查建议
catalog load 失败	catalog type / uri / env 错	先打印配置来源，不打印 secret
MinIO endpoint 连不上	容器内外 endpoint 不一致	区分 localhost 与服务名
bucket 不存在	warehouse root 不可写	先确认 bucket / access key
schema 不匹配	source of truth 不一致	回到 iceberg_schemas.py / mapping
重跑出现重复	dedupe / idempotency 不清	先看 key 和 report
inspect 没有 snapshot	只 ensure 表未写入数据	确认 append/overwrite 是否发生
add column 后写入失败	schema evolution 与数据不匹配	只做 add-column 边界演示

最小演示应该包含什么

这不是“跑过一次”的证明，而是状态闭环的验收清单。

catalog 能加载

说明入口有效。

namespace 能 ensure

bronze / silver 空间可用。

4 表 schema 能 ensure

schema source of truth 生效。

至少 1 Bronze + 1 Silver append

形成真实 snapshot。

snapshots / history / files 能看到

状态证据链存在。

add-column demo 有记录

schema evolution 可解释。

Pylceberg 本地闭环 API 流

用最小 API 流把概念压成可执行路径。

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline



API 的目的不是熟悉函数名，而是让 table state 变得可见。

add-column schema evolution 演示怎么讲

先读现有 schema

确认 latest schema id。

update_schema().add_column()

通过 Python API 增加字段。

再写一批数据

验证新字段可读。

记录 evidence

schema_evolution_demo_notes.md。

边界提醒

rename/drop/update 等复杂变更不作为本周主线。

APPEND VS OVERWRITE

append / overwrite 的边界：不要无解释覆盖历史

写入动作必须能在 report 中解释。

动作	适合场景	风险提示
append	新批次追加到 Bronze 或新增事实状态	重复输入要靠 dedupe/idempotency 解释
overwrite	可控重写目标状态或分区	不要无解释覆盖历史
schema evolution	字段边界内演进	必须记录兼容性说明
time travel check	写入后回看旧状态	依赖 snapshot 保留策略
baseline report	记录写入和 inspect 证据	没有 report 的成功不算验收

INSPECT OUTPUTS

history / snapshots / files 分别看什么

这些 inspect 结果会进入 Lesson05 baseline report。

snapshots

几次状态提交，每次 operation 是什么。

history

哪个 snapshot 何时成为 current。

files

当前状态引用多少 files，大小与分区如何。

metadata log

metadata 文件如何演进。

结论

不要各看各的，要组合解释。

RUNTIME PLAN

catalog_runtime_plan_v1.md 应该写什么

环境与配置来源

catalog type、uri、warehouse、endpoint。

表与 namespace

bronze / silver + 最小 4 表。

materialization 目标动作

ensure / write / inspect / report。

风险与占位

哪些命令未落地，何时同步。

runbooks/week04/README.md 主干

如何启动环境

compose / devbox / env 文件。

如何检查 catalog

加载、namespace、table。

如何 materialize

目标表、dry-run、写入边界。

如何 inspect

snapshots/history/files。

如何处理失败

按错误层级排查。

如何写 report

输出到 reports/week04。

官方资料怎么支撑本课判断

这页不是工具介绍会，而是把官方实践收口到 Week04 的工程判断。

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

Pylceberg

Python implementation 可在无需 JVM 的情况下访问 Iceberg tables。

Catalog-centric

Pylceberg 通过 catalog 组织和访问表，配置可从 YAML/env/API 加载。

Schema evolution API

`update_schema().add_column()` 可演示 add-column。

Reliability

Iceberg 提交通过 atomic metadata swap，支持可靠读与历史。

ANTI-PATTERNS

本课最容易做错的 5 件事

这些错误会让实操教学变成“本机偶然成功”。

错误	表面上	真正风险
硬编码 endpoint / secret	跑起来很快	换环境不可复现
只 ensure table, 不写数据	表存在	没有 snapshot 证据
只看 append 成功	写入成功	不知道 history/files 是否可检查
Dagster 先重构很厚	看起来平台化	核心 Pylceberg 逻辑难排错
伪造命令或输出	PPT 很完整	学生落地会失败

怎么证明 Week04 已经验收?

下一讲用 files / history / snapshots 做 baseline report, 而不是急着调优。

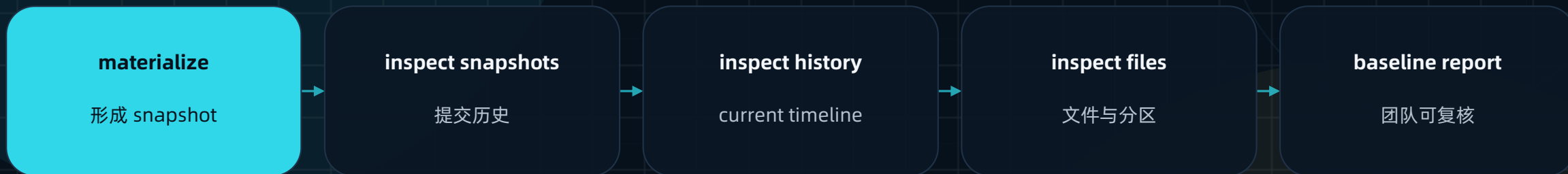
01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline



没有 baseline, 就没有真正优化, 只有模糊感受。

把 Week04 跑成本地最小闭环

本课最重要的 8 个判断

- Week04 不做重型基础设施秀。
- Pylceberg + SQL Catalog + MinIO 是当前教学最稳路线。
- Catalog / Warehouse / Table Location 必须拆开。
- materialization 不只是写入成功，而是形成可 inspect 的 snapshot。
- dry-run / plan / validation / idempotency 是最小防线。
- Dagster 本周只做 thin wrapper。
- 不伪造命令，不伪造运行结果。
- 下一讲用 files / history / snapshots 做 Week04 验收 baseline。

下一讲

性能基线不是调优冲动：files / history / snapshots 视角下的 Week04 验收。

课后最小行动

把今天的判断写进 repo 工件；不要只截图证明“看过”。

BASELINE · FILES · SNAPSHOTS

5. 性能基线不是调优冲动

没有 **baseline**，就没有真正的优化；
只有一堆模糊感受。

用 row count、snapshot count、file count、file size、partition distribution、history 和 metadata log 收口 Week04 验收。

baseline

files

history

snapshots

maintenance

handoff

Week04 学习链

01

Memory

为什么 AI 数据工程需要 “有记忆的表”

02

State Model

snapshot、manifest、metadata log 与 time travel

03

Bronze / Silver

Week03 入湖如何站成最小 4 表

04

Pylceberg

Catalog、Warehouse、写入、历史查看与 Schema Evolution

05

Baseline

files / history / snapshots 视角下的 Week04 验收

这节课先解决什么问题

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

为什么它重要

- Week04 到最后很容易被带偏成“开始调优”。
- 但本课真正收口是：在当前最小 Lakehouse 闭环里，看哪些对象才能说已经建立性能与状态基线。
- baseline 是团队协作的状态证据，不是 benchmark，也不是 tuning。

学完这节课，你至少应该能做到

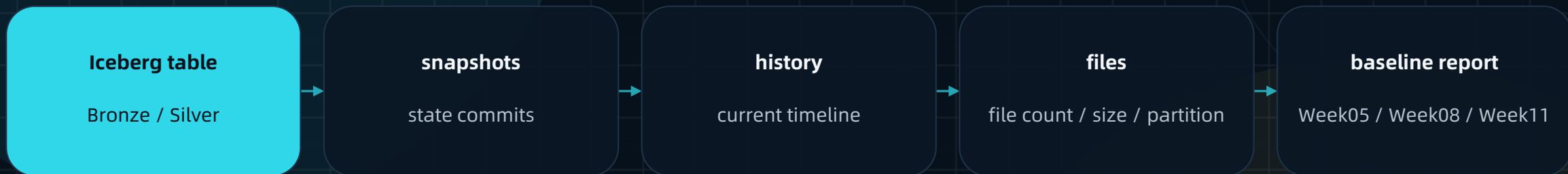
- 解释 baseline / benchmark / tuning 的差异。
- 列出 Week04 必须记录的指标。
- 把 snapshots、history、files、metadata log 组合解释。
- 写出 iceberg_baseline_report.md。
- 知道什么时候才考虑 expire snapshots / orphan cleanup / compaction。

本课产出：reports/week04/iceberg_baseline_report.md + runbooks/week04/baseline_inspection_notes.md。

LECTURE DIAGRAM · REDRAW

Iceberg table → inspect → baseline report

基于讲义图重绘：Iceberg table 的 snapshots / history / files / metadata log 进入 baseline script 和 markdown report。



Week04 验收不是 “用了 Iceberg” ，而是 “有可复核的数据状态基线” 。

Week04 的验收不是 “我们用了 Iceberg”

真正的验收是：团队已经建立第一份可以被复核的数据状态基线。

状态证据

snapshots / history / files。

性能观察

file count / size / partition。

团队复核

report 解释异常与下一步。

BASELINE VS BENCHMARK VS TUNING

Baseline 不是 benchmark, 也不是调参

先记录当前表状态, 再决定要不要优化。

概念	目标	Week04 是否主做
Baseline	记录当前状态: 表、files、snapshots、history、schema evolution	主做
Benchmark	测性能上限: 吞吐、延迟、并发能力	不主做
Tuning	改变系统行为以获得更好表现: compaction、partition、cache	不主做
Maintenance	控制 snapshots、metadata、orphan files、small files	只讲边界, 不做主线

Week04 应该记录哪些指标

指标	小白解释	异常信号
row count	当前表多少行	与 source coverage 不一致
snapshot count	表状态提交了几次	过多/过少都要解释
file count	当前状态引用多少 data files	文件特别碎
avg/min/max file size	文件大小分布	小文件过多或极端不均
partition distribution	分区分布	极不均匀或不符合查询模式
latest snapshot time	最新提交时间	与 ingest / release 时间对不上
metadata log entry count	metadata 演进记录数	保留策略可能不足

files / history / snapshots 怎么一起解释

snapshots

表状态提交了几次，每次 operation 是什么。

history

哪个 snapshot 在什么时候成为 current。

files

当前 snapshot 实际引用多少 files、大小和分区如何。

metadata log

metadata 状态如何演进。

组合口径

这次写入产生哪个 snapshot？当前引用哪些 files？是否与 source coverage 对得上？

像运维和数据 owner 一样读 baseline

先看表清单

4 表是否都形成状态。

再看 snapshot

本周有几次提交。

再看 files

是否过碎、过小、分布异常。

再看 schema evolution

add-column 是否记录证据。

最后写结论

哪些异常本周修，哪些留给后续。

MAINTENANCE BOUNDARY

什么时候才考虑维护动作

动作	什么时候考虑	Week04 口径
expire snapshots	历史 snapshot 太多，且已有保留策略	只讲边界；过期会缩短 time travel 历史
orphan file cleanup	出现未被 metadata 引用的遗留文件	只记录风险，不作为主线
compaction	小文件明显影响读取或维护	不要急着调优，先有 baseline
metadata cleanup	metadata 文件增长影响维护	记录 metadata log 和保留策略

Baseline report 模板

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

TERMINAL · STRUCTURED HANDOFF

Week04 Iceberg Baseline Report

1. 环境与运行说明

- repo commit:
- compose profile:
- catalog:
- warehouse:
- run timestamp:

2. 目标表清单

- bronze.raw_ticket_event
- bronze.raw_doc_asset
- silver.ticket_fact
- silver.knowledge_doc

3. 表状态摘要

| table | row_count | snapshot_count | file_count | latest_snapshot_time |

环境先写

没有环境，结果不可复核。

4 表清单

先验收最小闭环。

指标摘要

不要只截图，要有表格和解释。

GOOD VS INVALID REPORT

优秀 report 和无效 report 的区别

类型	内容特征	讲师判断
优秀 baseline report	有环境、命令/输出来源、指标、异常解释、下一步建议	可交接给团队
合格 baseline report	覆盖 4 表、snapshot、files、history、time travel、schema evolution	能说明 Week04 闭环存在
无效 baseline report	只有截图；只有命令没有结果；只有“成功了”没有状态证据	不能验收

ABNORMAL SIGNALS

异常怎么解释：先记，不修

先记录当前状态，再决定后续动作。

现象	可能原因	本周是否修
snapshot 特别多	频繁小批写入 / 重试 / 多次 demo	记录；不急着 expire
文件特别小	每次 append 数据量太小	记录；不急着 compaction
row count 和源表不一致	mapping / dedupe / missing handling 问题	优先解释
schema id 变化但无记录	schema evolution 未写 notes	本周修文档
partition 分布极不均匀	分区策略不服务查询模式	记录，后续演进
time travel 读不到旧 snapshot	snapshot 过期或未形成旧状态	先解释保留边界

文件特别小不要急着 compaction

01

多次小批 append

demo / retry

02

snapshot 增长

状态很多

03

files 过碎

file_count 高

04

读性能担忧

想立刻 compaction

05

先 baseline

记录再决策

屏幕上看起来像什么

- file_count 很高
- avg file size 很小
- 学生想马上优化

实际已经坏在哪

- 没有 baseline 之前无法判断优化是否有效
- compaction 会改变文件布局，需要记录
- Week04 主线不是 tuning

snapshot 特别多不要盲目 expire

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

01

频繁写入

很多 snapshots

02

担心 metadata

想清理

03

expire snapshots

旧状态删除

04

time travel 失败

旧评测无法复现

05

策略缺失

团队争论

屏幕上看起来像什么

- metadata 变小
- 最新表仍可读
- 似乎维护成功

实际已经坏在哪

- 旧 snapshot 不再可用于 time travel
- 评测和 release 复盘可能断
- 必须先有保留策略和报告记录

time travel 读不到旧 snapshot

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

01

bad case 出现

要回到旧数据状态

02

查 snapshot

目标 snapshot 不在

03

查 history

只剩近期状态

04

查 cleanup

被过期

05

复盘失败

只能猜

屏幕上看起来像什么

- 当前表没坏
- 最新查询正常
- 团队容易以为没问题

实际已经坏在哪

- 历史状态不可回看
- 说明 baseline 和 retention 没设计好
- 不能只看当前表是否能读

衔接 Week05 / Week06 / Week08

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

Week05 transform

知道自己消费哪版 Silver。

Week06 asset factory

materialization 和 baseline 证据要挂起来。

Week08 retrieval consistency

索引对应哪版文档资产。

Week11+ eval / release

绑定数据状态，而不只绑定代码。

Week04 全周收官图

Memory → State Model → Bronze/Silver → PyIceberg → Baseline。

01 Memory

02 State Model

03 Bronze/Silver

04 PyIceberg

05 Baseline



Week04 的结果不是用上 Iceberg，而是团队拥有第一份可复核的数据状态基线。

Week04 最终交付

01 Memory

02 State Model

03 Bronze/Silver

04 PyIceberg

05 Baseline

lakehouse_foundation_v1.md

Week04 为什么存在。

source_to_iceberg_mapping_v1.md

source → Iceberg 字段映射。

bronze_silver_table_design_v1.md

最小 4 表设计。

catalog_runtime_plan_v1.md

catalog / warehouse / table location。

runbooks/week04/README.md

运行与排查路径。

iceberg_baseline_report.md

最终状态基线。

官方资料怎么支撑本课判断

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

Performance

manifest list 和 manifest files 支持 scan planning、metadata filtering 与 data filtering。

Maintenance

expire snapshots 控制 metadata 和文件规模，但旧 snapshot 不再可用于 time travel。

Reliability

snapshot history 支持 rollback，读者使用一致 snapshot。

Pylceberg API

inspect snapshots / history / files / metadata log 可作为 baseline report 来源。

Week04 自检清单

01 Memory

02 State Model

03 Bronze/Silver

04 Pylceberg

05 Baseline

4 表是否明确?

Bronze / Silver 最小表都能解释。

mapping 是否写清?

source → target / transform / dedupe。

snapshots 是否可 inspect?

有状态提交证据。

history/files 是否可解释?

不是只看成功。

baseline report 是否完整?

环境、指标、异常、下一步。

NOT TUNING YET

现在不要急着 tuning

冲动	为什么先别做	正确下一步
马上 compaction	没有 baseline, 不知道优化前状态	先记录 file size / file_count
马上 expire snapshots	可能删掉 time travel 历史	先写 retention 口径
马上改 partition	可能让教学范围膨胀	先记录查询模式和分布
马上接 Trino/Spark	会把主线带到平台栈	先让 Pylceberg 机制可见

FINAL WORD

baseline 先于 tuning

不要说“用了 Iceberg”

这不是验收标准。

要说“状态证据是什么”

snapshot / history / files / schema evolution。

不要只有截图

截图不能交接状态。

要有报告

baseline report 是团队协作证据。

从 ingest baseline 升级成 Lakehouse state baseline

本课最重要的 8 个判断

- Baseline 先于 tuning。
- Week04 验收不是“用了 Iceberg”，而是“有可复核状态证据”。
- row count、snapshot count、file count、file size、partition distribution 必须被记录。
- snapshots、history、files 要合起来解释。
- expire snapshots / compaction 不是 Week04 主线。
- baseline report 不能只有截图。
- Week05 / Week06 / Week08 会持续消费 Week04 状态基线。
- Week04 把 Week03 ingest baseline 升级成了 Lakehouse state baseline。

Week05 预告

Transform 与语义层会站在稳定 Bronze / Silver 状态上继续推进。

课后最小行动

把今天的判断写进 repo 工件；不要只截图证明“看过”。

Week05 将从这份 baseline 出发



课程主线继续：每一周都不是孤立工具，而是在给 AI 系统补一层生产能力。